

1. 事業領域 (ビジネスドメイン) ... 事業を展開する領域 (ニサービス種)
 業務領域 (サブドメイン) ... ビジネスドメインを細分化したものである (ニサービスの施策種) (ニ時間と労力は)
 ↳ 中核 ... 競合他社との差を生み出す部分、参入障壁は高いことが理想 (表1.1)
 ↳ 一般 ... ECサイト内の認証処理などのような、どの会社も同様の手法をとる領域
 ↳ 補完 ... 中核を補佐するが、競争優位性を生み出すわけではない領域、なるべく低コストに
 こからは、ユーザーが分離しようとすると考えやすい
 業務エキスパート ... 業務領域での活動に精通している人

2. 開発者は、業務課題とその背景を理解する必要があり (理解している事項のみ、リリースコードに反映させる)
 ⇒ 業務エキスパートから開発者への情報伝達と、情報の伝落なく行うことが重要 (あいまいさや同義表現は排除)
 ↳ この基準の「同じ言葉 (ユニバース言語)」を育成すると良い (あくまで業務用語、×技術用語)
 モデル ... 現実のモデルや出来事と抽象的・簡略的に表現したもの (目的に合ったものだけを提供するのは)
 ⇒ 業務ロジックの共有 車両のwikiやテストコードなどモデルに依存する

3. 同じ言葉に複数の解釈の仕方があり ⇒ その多様性をソフトウェアで表現することはほぼ不可能
 ↳ 複数の意味で使用しない (～な客など) モデルは、その文脈の中で使うことが必要
 ↳ 区切りが文脈に割り合える (モデルを作るために設計した環境によって異なるニスコ-プ)
 業務領域は「発見」であり、区切りが文脈は「設計」⇒ ソフトウェア開発者の区切りが文脈と定義する
区切りが文脈はモデルの境界を定義 → システム設計の定義 → アーキテクチャの選択
 ↳ 同一区切り単位ごとに、別々のモデルが存在する

4. 区切りが文脈同一の連携
 ↳ 良きパートナー ... 双方向の情報交換
 ↳ モデルの共有 ... コストと効果を考えながら、慎重に融通を効かせる
 ↳ 利用者と供給者の関係
 ↳ 従属関係 ... 下流から上流に従う
 ↳ モデル変換装置 ... 利用者側のフォーマットを行う
 ↳ 共用サービス ... 供給者側が一定のフォーマットを行う (公開された言葉に合わせる)
 ↳ 互いに独立 ... コストや効率を考えると別々で行う

5. 業務ロジックが単純な場合には適した実装 分散トランザクションは危険 (必要性重要)
 ↳ トランザクションスプリング ... 失敗してもロールバックをするが、データの一意性を保てさせるためのスプリング
 ↳ アテイングレコード ... モデル上のオブジェクトとテーブル構造とのマッピングをカプセル化したもの
 cf) ORM: Object Relational Mapping

6. ドメインモデル ... ロジックとデータの両方を一体化させたオブジェクトモデル (業務ロジックに焦点を合わせる)
 ↳ 値オブジェクト ... 値の妥当性をチェックやカプセル化を行うためのオブジェクト (クラス) コードの意図も明確になる
 ↳ 集約 ... データの一意性を強制する。集約内部のインターフェースのみが状態を管理する ← 公開インターフェースは明示的に
 ↳ 業務サービス ... 業務ロジックのみを記述する。自分自身の状態は持たないオブジェクト → コマンド

現在の状態に関する情報を業務イベントとして表現する ⇒ 業務プロセスの最適化

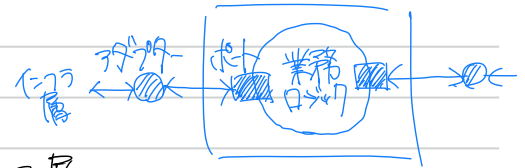
7. イベント履歴式ドメインモデル... イベントソーシング"を用い、集約の状態を管理
↳ 時間軸の視点を取りとけたデータモデル
ステータスやバージョンとして管理する
イベントストア... イベント情報を永続化するデータベース (記録したイベントの変更や削除はできない)

8. レイードアーキテクチャ... 実装上の関心事と切り離すアーキテクチャ。依存の方向はトップダウン
(
・プレゼンテーション層... プログラムの公開インターフェース (WebUI, CLI, RestAPI)
・サービス層... 業務ロジック層のフサード (コントローラ, アクション)
・業務ロジック層... 業務ロジックをカプセル化する層 (エンティティ, 集約) S3やCloudStorageなど
・データアクセス層... データの永続化と接続 (DB, メールサーバ, オブジェクトストレージ)

これはトップダウンへの依存であり、依存関係逆転の原則に反する

⇒ ポートとアダプター (ハキサゴナルアーキテクチャ)

- (
・業務ロジック層... 同上
・サービス層... 同上
・インフラストラクチャ層... プレゼンテーション層 + データアクセス層



コマンド・リクエスト責任分離... モデルの責任範囲を2種類に分離する

- (
・コマンド実行モデル... システムの状態を変更する操作 (コマンド) をもつモデル
・読み取りモデル... データの投影結果のやり取りをするモデル (基本的には、同期投影を行う)

9. モデル変換... モデル間での通信をすなわち、プロキシやAPIゲートウェイを用いて変換すること
トランザクションの整合性を保つ工夫として、
・送信箱
・サーガ
・プロセスマネージャ

同じ集約に所属する
業務プロセスに強い
一貫性を保証することに寄与

10. 複数の区切りがたて文脈に対処する変更は工数がかかりやすい
⇒ 複数の業務領域を含むおに広い範囲で文脈を区切ることOK (中核基盤で)
実装方針の策定については、図10-7にフローチャートがあり、それを参照のこと

11. 中核、一般、補完の間での業務領域カテゴリの変化は十分起こりうる
↳ この格好変更の場合、設計の再検討が必要かも?

トランザクションスクリプト → アクティブルコード → ドメインモデル → イベント履歴式ドメインモデル
のように昇華させることは可能

システムが成長することは良いことである一方、無秩序に成長させると大きな泥団子になってしまう

1. 事業活動と業務領域の分析をする

事業価値の切り口と境界を引くには1つの手

2. モデルをつくる

文脈の境界を時々見直し、モデルを単純に保つことが重要

3. モデルに基づいて、ソフトウェアを設計する

「生において唯一不変なことは変化することである」

12. イベントストリーミング... モデルの村界とする範囲 (業務プロセス) を決めするためのグレイテストリーミング
一連の業務イベントを付箋を用いて並べながら探る
⇒ 同じ言葉と集めること、文脈を適切に区切ること、モデリング" などを同時に行うことが必要

・中核領域の刷新には注意
・年次作業知識をもしあが回収

13. 実際上のドメイン駆動設計

1. 戦略的な分析 ... 業務領域の分析, 既存システムの構造解析

2. 設計改善 ... 論理的な境界と業務領域の境界は対応させる

- ・ ストラテジー方式 ... 新システムに段階的に移行していく方式。既存と新システムが一時的に共存
- ・ リファクタリング

→ 公開インターフェース

14. サービス ... インターフェースを用いて, システム上の機能を提供するもの

マイクロサービス ... サービスを分割し, 個々のサービスの役割を減らしたもの

- ↓ ↑ ×
 - ・ 細かくする, 適切な粒度で行うことが必要 (分散化を泥団子にしない)
 - ・ 深いモジュール (機能の幅は狭く, 業務ロジックを多く含む) を目指すべき
- 「どのように」ではなく, 「何を」に注目する。自然と深いモジュールになることが多い

15. イベント駆動型アーキテクチャ ... システムのコンポーネント間では, イベントメッセージを非同期でやりとりする技術方式

- ・ イベント通知 ... イベントが発生したという事実のみを送信
- ・ イベントによる状態転送 ... イベントの詳細は何にせよ, それにより変化した状態を送信
- ・ 業務イベント ... イベントの詳細情報を送信

設計する際には, 最悪のケースとして想定しておくことと回避し, 一貫性を保つことが重要

16. OLTP (オンライントランザクション処理) ... 業務系データモデルが対象とするもの (パフォーマンス最適化?)

OLAP (オンライン分析処理) ... 分析系データモデルが対象とするもの

- ・ 事実テーブル ... 業務イベントを反映して, 現在の状態を保持していく
 - ・ 特性テーブル ... 事実の性質や状態を記述するテーブル (形容詞?)
- ⇒ 星形スキーマ, 雪の結晶形スキーマというテーブル構造をもちこきと良い

- 分析系データの管理基盤
- ・ DWH (データウェアハウス) ... システムからデータを抽出し, 分析系モデルに変換するためのもの
 - ・ データレイク ... 業務系モデルのデータの状態のデータを取り込めるもの (生データ)
 - ・ データマート ... 特定の目的に合わせて, DWH からデータを一部抽出したもの

データレイクの4つの基本原則

- ・ 業務視点で分割する ... 区切りが作業脈と一致させる
- ・ データをプロダクトと考える ... 使い手側が容易に利用できるように設計にする
- ・ 自律性を高める ... データプロダクトの管理基盤を構築する
- ・ エコシステムを構築する ... 管理におけるガバナンスを策定する

